



School of Computer Science

COMP 8347 “Internet Applications and Distributed
Systems”

Summer 2026

Week 5

Authentication, Authorization, and Security

DR. OLENA SYROTKINA



University of Windsor

Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- Django Authentication Basics (User model, login/logout)
- Authorization & Permissions
- Security Practices (CSRF, XSS, password hashing, HTTPS)

Django Authentication Basics

Unlike **Student** (which we created), Django already provides a ready-to-use User model inside **django.contrib.auth**

It has the following fields:

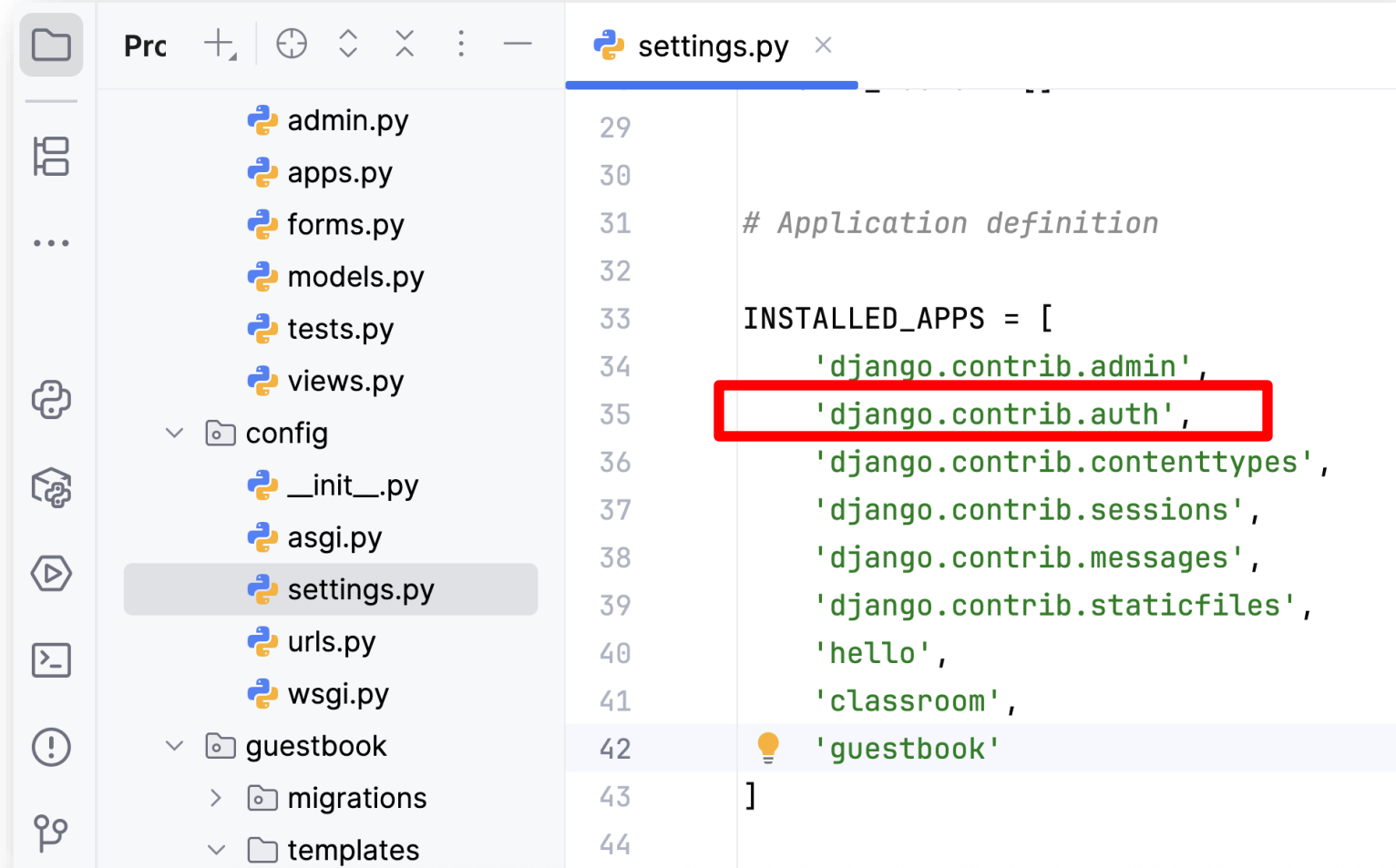
- username → required unique identifier (by default).
- password → securely stored (hashed, not plain text).
- email → optional, but often required in real projects.
- first_name / last_name → profile fields.
- is_staff → can log into the admin site.
- is_superuser → bypasses all permission checks.
- is_active → controls whether account is active.

Django Authentication Basics

How do we get all this?

The User model comes from the **auth app**, which is already in **INSTALLED_APPS** inside **settings.py**.

After we ran python **manage.py migrate** weeks ago, Django created a database table for users automatically (auth_user).



```
29
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'hello',
41     'classroom',
42     'guestbook'
43 ]
44
```

Django Authentication Basics

A) How do we get all this?

You can see it in your DB with:

```
python manage.py dbshell
```

```
.tables
```

```
SELECT * FROM auth_user;
```

B) Creating the First User (We have one! Remember? See your lecture from Week 3)

```
python manage.py createsuperuser
```

It will ask:

- ✓ Username:
- ✓ Email address:
- ✓ Password:

Username: SuperAdmin

Email: SuperAdmin@uwindSOR.ca

Password: SuperPass123

```
python manage.py runserver
```

Log in at <http://127.0.0.1:8000/admin/> using the username and password you just set.

Django Authentication Basics

C) Adding Login/Logout Pages to Your Site

In config/urls.py, add:

```
from django.urls import path, include

urlpatterns += [
    path("accounts/",
include("django.contrib.auth.urls")),
]
```

This line gives us a lot of URLs for free:

- ✓ /accounts/login/
- ✓ /accounts/logout/
- ✓ /accounts/password_change/

Django Authentication Basics

D) Creating Login Template

Django looks for registration/login.html.
Create this file:

templates/registration/login.html

```
{% extends "classroom/base.html" %}
{% block content %}
<div class="page">
  <h1>Login</h1>
  <form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Log In</button>
  </form>
</div>
{% endblock %}
```

Open config/settings.py

Make sure Django knows where to look for templates

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / "templates"],
        'APP_DIRS': True,
        'OPTIONS': {
```

Now restart server, go to:
<http://127.0.0.1:8000/accounts/login/>

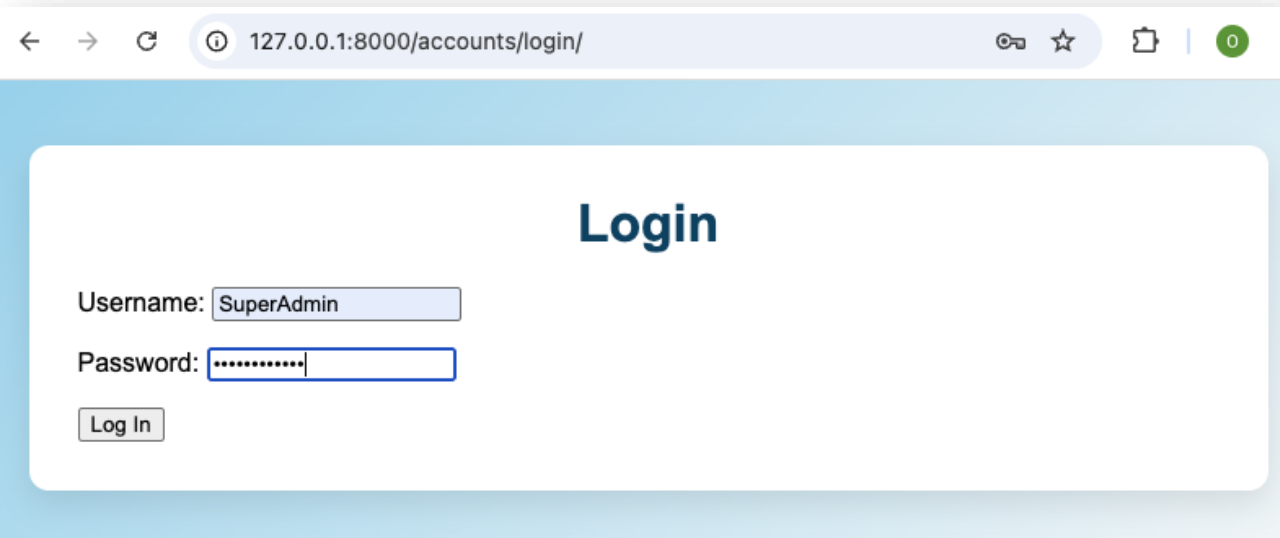
You'll see a login form.

Try logging in with your superuser credentials.

Django Authentication Basics

Go to:

`http://127.0.0.1:8000/accounts/login/`

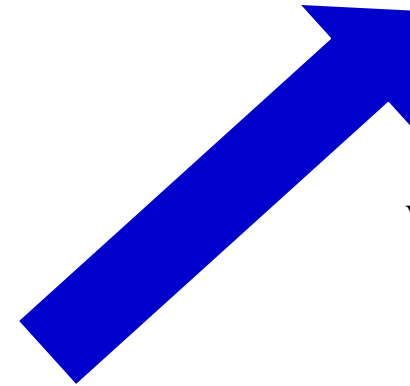
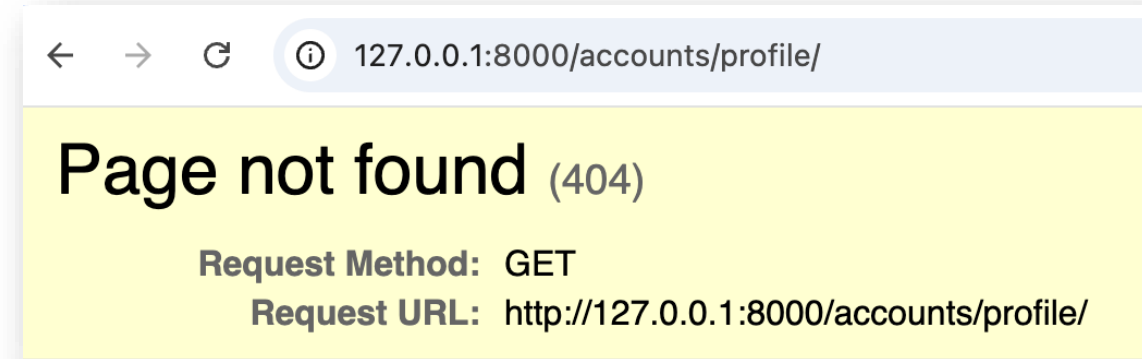


Login

Username:

Password:

Press the button and you will see...



We need to create a profile view!

Django Authentication Basics

E) Creating a Profile View

In your classroom/views.py, add:

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render

@login_required
def profile(request):
    return render(request, "classroom/profile.html", {
        "user": request.user
    })
```

- We use `@login_required` so only logged-in users can see it.
- We pass the `request.user` object into the template (Django automatically attaches the currently logged-in user).

Django Authentication Basics

F) Adding a URL for Profile

In config/urls.py:

```
from django.urls import path, include
from classroom.views import profile

urlpatterns += [
    path("accounts/", include("django.contrib.auth.urls")),
    path("accounts/profile/", profile, name="profile"),
]
```

- Now /accounts/profile/ is mapped to our new view.
- Next step is to create a **profile template**!

Django Authentication Basics

G) Creating the Template

Create file: templates/classroom/profile.html

```
{% extends "classroom/base.html" %}
{% block content %}
<div class="page">
  <h1>User Profile</h1>
  <p><strong>Username:</strong> {{ user.username }}</p>
  <p><strong>Email:</strong> {{ user.email }}</p>
  <p><strong>First name:</strong> {{ user.first_name|default:"(not set)" }}</p>
  <p><strong>Last name:</strong> {{ user.last_name|default:"(not set)" }}</p>

  <p><a href="{% url 'student-list' %}">Go to Student List</a></p>
  <p><a href="{% url 'logout' %}">Log out</a></p>
</div>
{% endblock %}
```

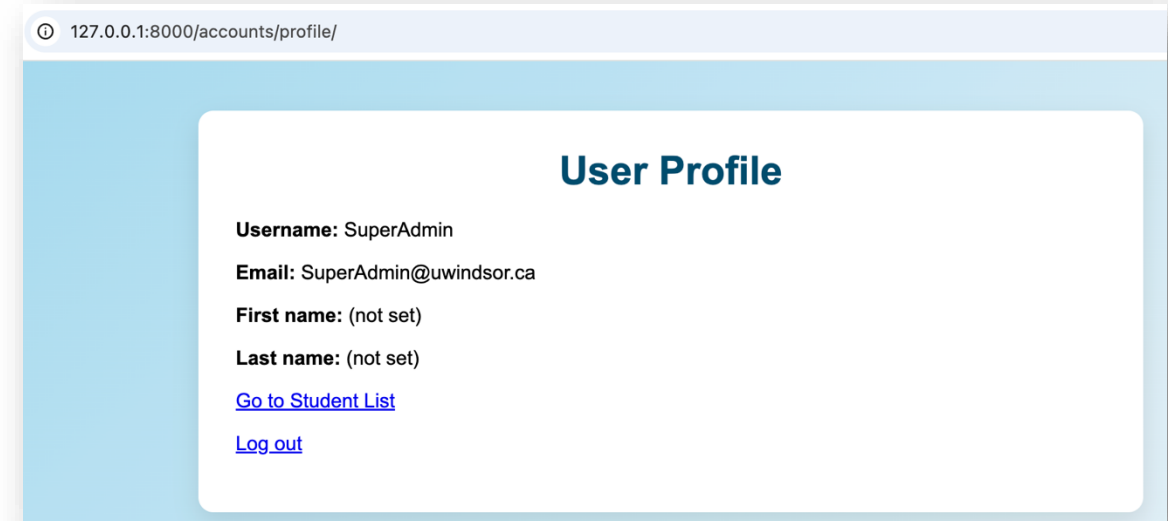
Django Authentication Basics

Let's test it:

Start your server:

```
python manage.py runserver
```

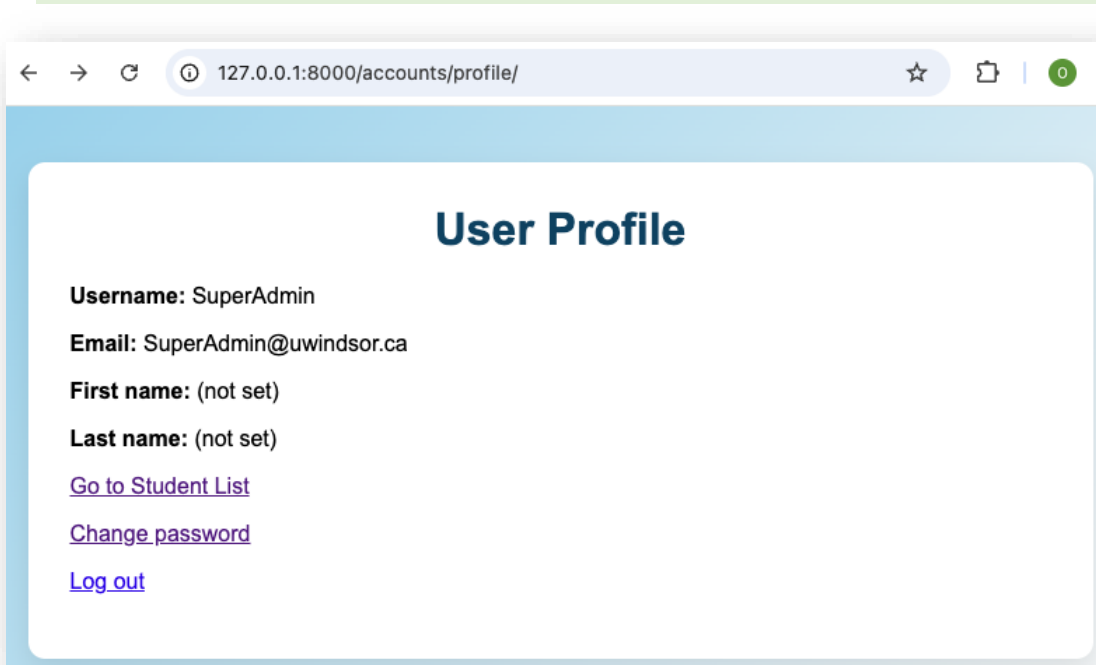
- ✓ Go to <http://127.0.0.1:8000/accounts/login/>
- ✓ Log in with your superuser.
- ✓ You will be redirected to /accounts/profile/.
- ✓ Now instead of a 404, you will see your profile page with your username/email/etc.



Django Authentication Basics

H) Let's add a "Change password" link – open `classroom/profile.html` and add:

```
{% if user.is_authenticated %}
    <a href="{% url 'password_change' %}">Change password</a>
{% endif %}
```



Now, when you're logged in and click **Change password**, Django shows the default form.

A screenshot of the Django password change form in a web browser. The browser's address bar shows '127.0.0.1:8000/accounts/password_change/'. The page has a dark blue header with 'Django administration' in yellow. Below the header, a breadcrumb trail reads 'Home > Password change'. The main content area is titled 'Password change' and contains the instruction: 'Please enter your old password, for security's sake, and then enter your new password twice so we can verify you typed it in correctly.' There are three input fields: 'Old password:', 'New password:', and 'New password confirmation:'. Below the 'New password:' field, there are four lines of feedback text: 'Your password can't be too similar to your other personal information.', 'Your password must contain at least 8 characters.', 'Your password can't be a commonly used password.', and 'Your password can't be entirely numeric.' Below the 'New password confirmation:' field, there is a line of feedback text: 'Enter the same password as before, for verification.' At the bottom of the form, there is a dark blue button labeled 'CHANGE MY PASSWORD'.

Django Authentication Basics

I) Where to customize the page (templates)

Django will look in your project for these templates (you create them):

- ✓ templates/registration/password_change_form.html
- ✓ templates/registration/password_change_done.html

If you don't create them, Django's default templates still work.

If you do create them, you can style/word them however you want.

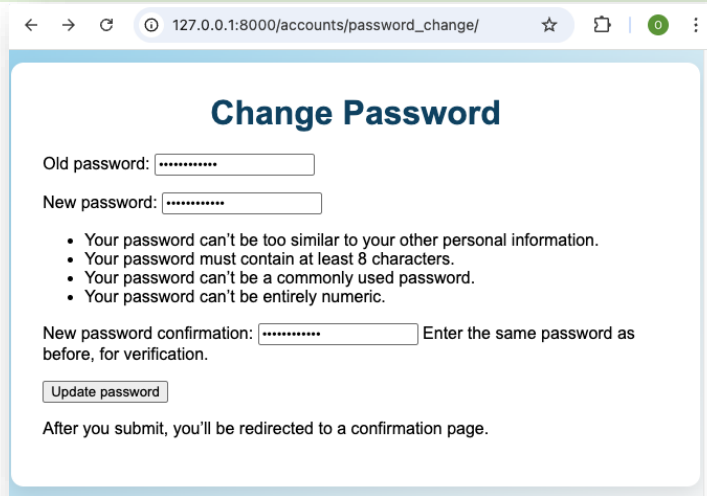
Example: templates/registration/password_change_form.html

```
{% extends "classroom/base.html" %}
{% block title %}Change Password{%
endblock %}
{% block content %}
<div class="page">
  <h1>Change Password</h1>
  <form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Update
password</button>
  </form>
  <p class="muted">After you submit,
you'll be redirected to a confirmation
page.</p>
</div>
{% endblock %}
```

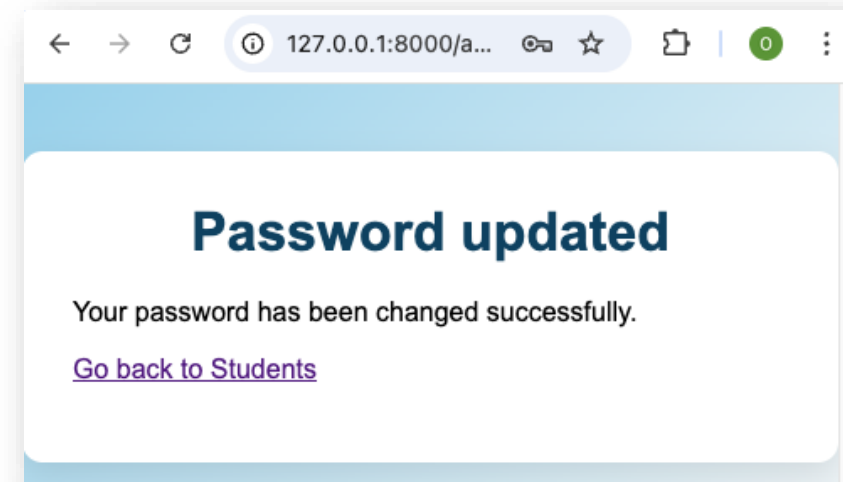
Django Authentication Basics

Example: `templates/registration/password_change_done.html`

```
{% extends "classroom/base.html" %}
{% block title %}Password Changed{% endblock %}
{% block content %}
<div class="page">
  <h1>Password updated</h1>
  <p>Your password has been changed successfully.</p>
  <p><a href="{% url 'student-list' %}">Go back to Students</a></p>
</div>
{% endblock %}
```



A screenshot of a web browser showing the Django password change form. The URL is `127.0.0.1:8000/accounts/password_change/`. The form has a title "Change Password" and includes fields for "Old password:", "New password:", and "New password confirmation:". Below the fields are four bullet points providing password requirements: "Your password can't be too similar to your other personal information.", "Your password must contain at least 8 characters.", "Your password can't be a commonly used password.", and "Your password can't be entirely numeric." There is an "Update password" button and a note stating "After you submit, you'll be redirected to a confirmation page."



A screenshot of a web browser showing the Django password change confirmation page. The URL is `127.0.0.1:8000/a...`. The page has a title "Password updated" and a message "Your password has been changed successfully." Below the message is a link "Go back to Students" in purple text.

Django Authentication Basics

K) Let's allow the user to log out. Go to templates/registration/logged_out.html

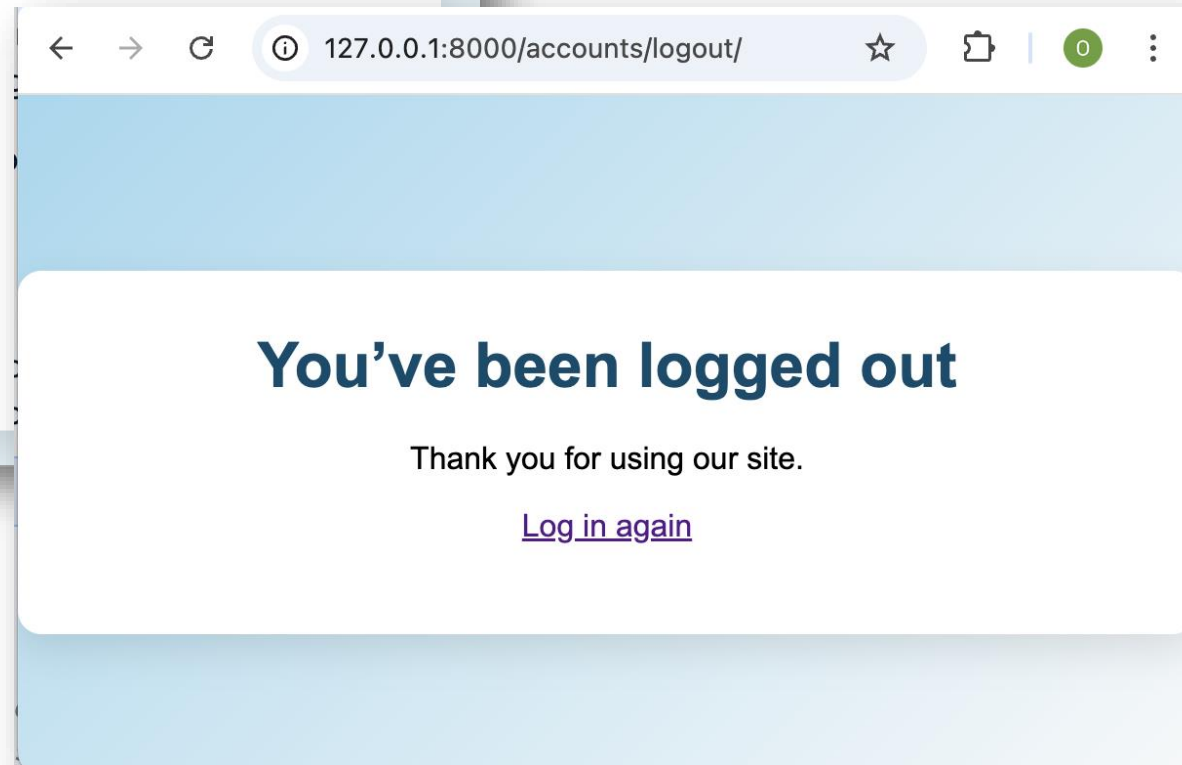
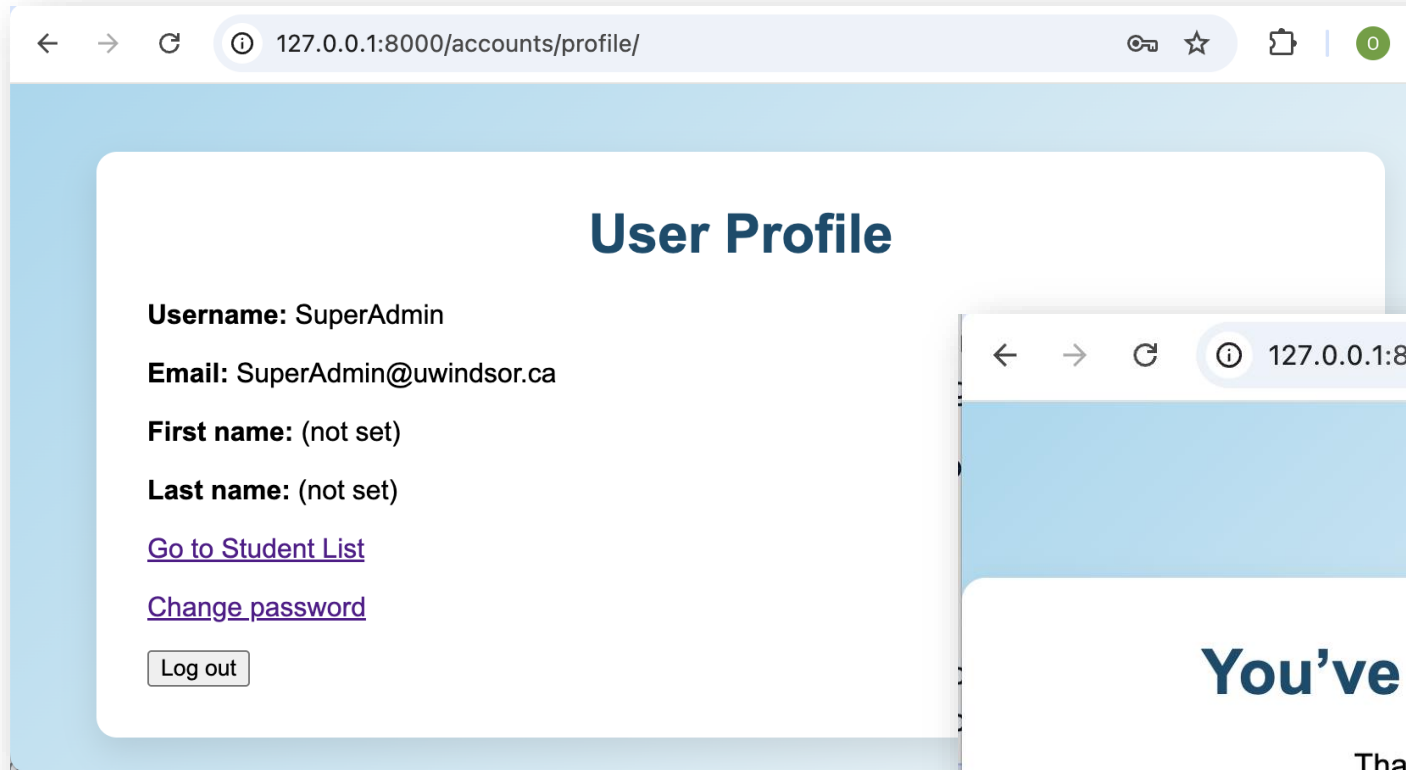
```
{% extends "classroom/base.html" %}
{% block title %}Logged Out{% endblock %}
{% block content %}
<div class="page" style="text-align:center; margin-top:50px;">
    <h1>You've been logged out</h1>
    <p>Thank you for using our site.</p>
    <p><a href="{% url 'login' %}">Log in again</a></p>
</div>
{% endblock %}
```

Django Authentication Basics

Now go to templates/classroom/profile.html and add the following code:

```
{% if user.is_authenticated %}
    <form method="post" action="{% url 'logout' %}"
style="display:inline">
    {% csrf_token %}
    <button type="submit">Log out</button>
</form>
{% else %}
    <a href="{% url 'login' %}">Log in</a>
{% endif %}
```

Django Authentication Basics



Authorization & Permissions

Concept	Meaning	Analogy
Authentication	Confirms who you are	Showing your student ID to enter the building
Authorization	Confirms what you can do	Being allowed to enter specific labs or offices once inside

Restricting access to pages

When you build web apps, not every page should be public. For example:

- /students/add/ should only be used by logged-in users.
- /dashboard/ might show private info.

To enforce that, Django provides the decorator.

Authorization & Permissions

Restricting access to pages

To enforce that, Django provides the decorator:

```
from  
django.contrib.auth.decor  
ators import  
login_required  
  
@login_required  
def add_student(request) :  
    ...
```

imports the function that checks if the user is logged in.

placed right above the view function you want to protect.
if a logged-out user visits this URL, Django redirects them to /accounts/login/.

Authorization & Permissions

Open `classroom/views.py` and update your view:

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect
from .models import Student
```

`@login_required`

```
def add_student(request):
    if request.method == "POST":
        name = request.POST.get("name")
        email = request.POST.get("email")
        Student.objects.create(name=name, email=email)
        return redirect("student-list")
    return render(request, "classroom/add_student.html")
```

Now, restart your server and:

1. Log out.

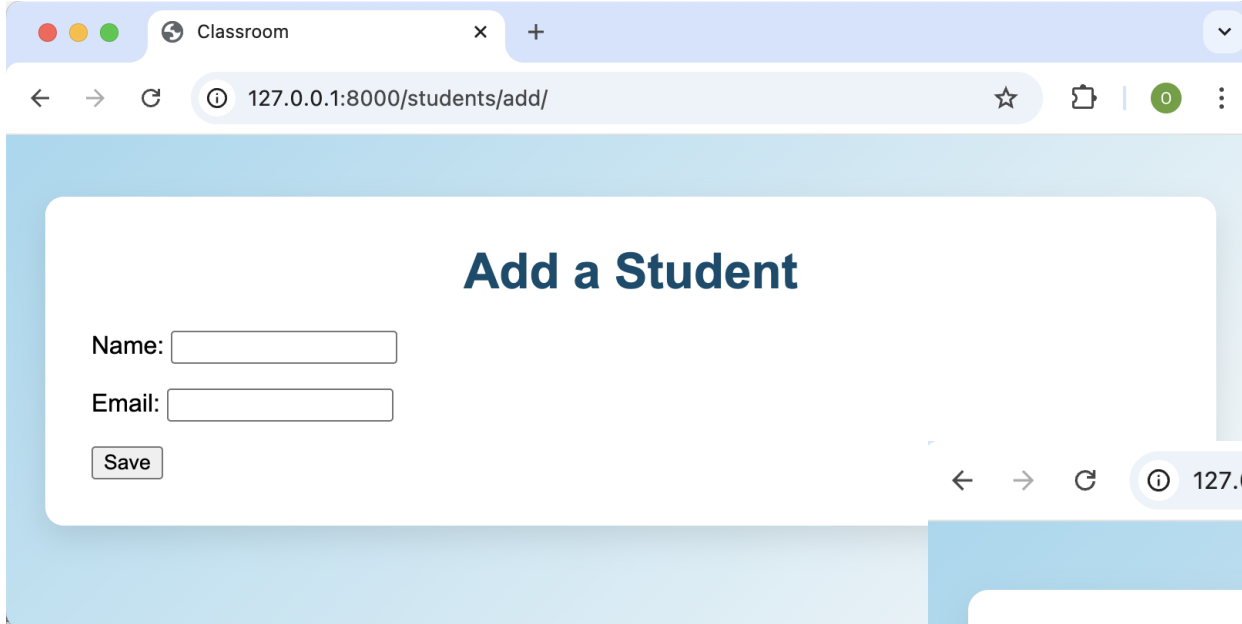
2. Try visiting `/students/add/`.

→ You'll be redirected to `/accounts/login/` automatically.

3. Log in and try again.

→ The page will open.

Authorization & Permissions



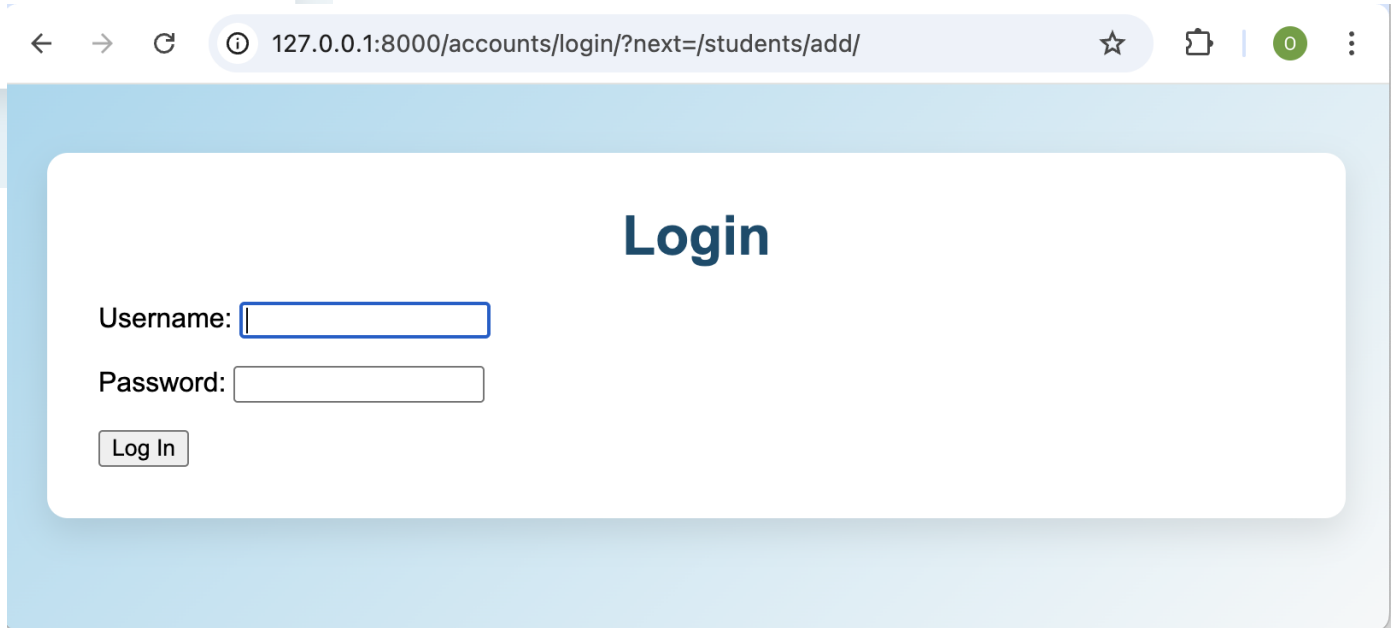
A screenshot of a web browser window. The tab is labeled 'Classroom'. The address bar shows '127.0.0.1:8000/students/add/'. The page title is 'Add a Student'. The form contains two input fields: 'Name:' and 'Email:'. Below the fields is a 'Save' button.

Add a Student

Name:

Email:

VS.



A screenshot of a web browser window. The address bar shows '127.0.0.1:8000/accounts/login/?next=/students/add/'. The page title is 'Login'. The form contains two input fields: 'Username:' and 'Password:'. Below the fields is a 'Log In' button.

Login

Username:

Password:

Authorization & Permissions

Now that we can require login, let's go deeper – not all logged-in users are equal.

Roles in real systems

Think about your university:

- **Admin (Instructor)** – can add or delete students.
- **Assistant (TA)** – can view students but not delete them.
- **Student** – can only see their own info.

Django has a built-in way to model that using:

- **Groups** (e.g. “Teachers” or “Students”)
- **Permissions** (e.g., “Can add student”, “Can delete student”)

Authorization & Permissions

Groups & Permissions in Django Admin

Step 1. Open Django admin

Go to: <http://127.0.0.1:8000/admin/>

Log in with your superuser credentials.

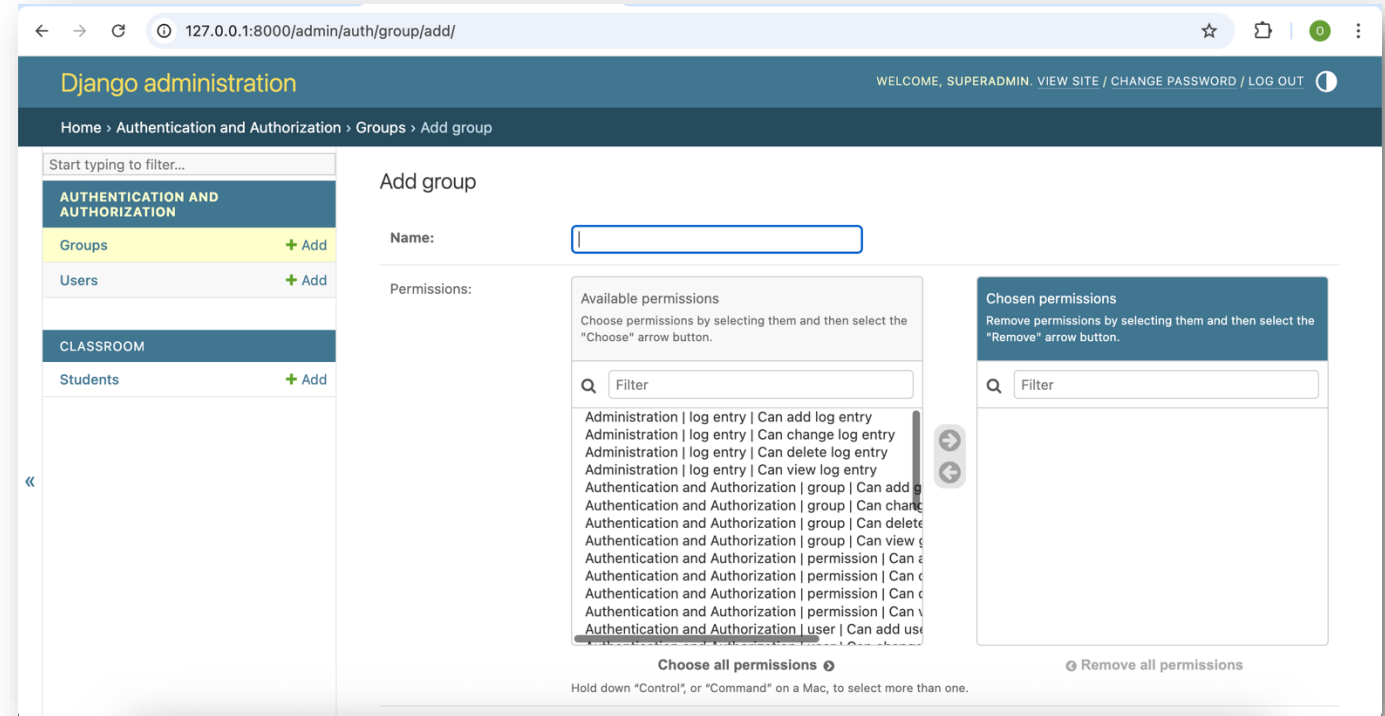
Step 2. Create a group

Click Groups → Add Group

Name it **Teachers**

Scroll to “Permissions” and check:

- ✓ Can view student
- ✓ Can add student
- ✓ Can change student
- ✓ Can delete student



Then click Save

Authorization & Permissions

Groups & Permissions in Django Admin

Create another group called Viewers (only check Can view student).

Step 3. Assign a user to a group

Go to **Users** → **select a user**

Scroll down to **Groups**

Add them to “Teachers” or “Viewers”

Save.

My users:

Name: Teacher

Password: T123each

Name: RegUser

Password: R123user

Authorization & Permissions

Checking permissions in views

Now let's protect your “add student” page so that only users with permission can access it. Add this to your **views.py**:

```
from django.contrib.auth.decorators import permission_required

@login_required
@permission_required('classroom.add_student', raise_exception=True)
def add_student(request):
    ...
```

- 'classroom.add_student' → "appname.permissioncodename".
- If the user **doesn't** have permission, Django will return **403 Forbidden**.
- If they do, the view runs normally.

Authorization & Permissions

Dashboard Overview

Let's give logged-in users one place to see key info such as:

- ✓ Greeting (“Welcome, John!”)
- ✓ Quick stats (number of registered students)
- ✓ Useful links (View Students, Add Student, Change Password)
- ✓ Role display (show if user is Admin/Teacher/Viewer)

Create a Dashboard View

Open `classroom/views.py` and add:

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render
from .models import Student

@login_required
def dashboard(request):
    student_count = Student.objects.count()
    user = request.user
    context = {
        "student_count": student_count,
        "user": user,
    }
    return render(request,
"classroom/dashboard.html", context)
```

Authorization & Permissions

Add URL Pattern

In `classroom/urls.py`, add:

```
from django.urls import path
from . import views

urlpatterns = [
    path("dashboard/", views.dashboard,
name="dashboard"),
    path("students/", views.student_list,
name="student-list"),
    path("students/add/",
views.add_student, name="add-student"),
]
```

Then make sure `config/urls.py` includes:

```
path("", include("classroom.urls")),
```

Open <http://127.0.0.1:8000/dashboard/>

Create Template: dashboard.html

File: `classroom/templates/classroom/dashboard.html`

```
{% extends "classroom/base.html" %}
{% block title %}Dashboard{% endblock %}
{% block content %}
<div class="dashboard-container">
    <h1>Welcome, {{ user.first_name|default:user.username }}!</h1>

    <div class="stats-box">
        <p><strong>Total Students:</strong> {{ student_count }}</p>
    </div>

    <div class="quick-links">
        <h2>Quick Links</h2>
        <ul>
            <li><a href="{% url 'student-list' %}">View Students</a></li>
            {% if perms.classroom.add_student %}
            <li><a href="{% url 'add-student' %}">Add Student</a></li>
            {% endif %}
            <li><a href="{% url 'password_change' %}">Change
Password</a></li>
        <p></p>
        <form method="post" action="{% url 'logout' %}"
style="display:inline">
            {% csrf_token %}
            <button type="submit">Log out</button>
        </form>
    </ul>
</div>
</div>
{% endblock %}
```

Authorization & Permissions

Optional: Add this to your static/classroom/styles.css (or wherever your CSS lives):

```
.dashboard-container {  
  max-width: 700px;  
  margin: 50px auto;  
  background-color: #f8f9fc;  
  border-radius: 10px;  
  padding: 30px;  
  box-shadow: 0 4px 8px rgba(0,0,0,0.1);  
}  
  
.stats-box {  
  background: #e9f5ff;  
  border-left: 5px solid #0066cc;  
  padding: 15px;  
  margin: 20px 0;  
}
```

```
.quick-links ul {  
  list-style-type: none;  
  padding: 0;  
}  
  
.quick-links li {  
  margin: 8px 0;  
}  
  
.quick-links a {  
  color: #0066cc;  
  text-decoration: none;  
}  
  
.quick-links a:hover {  
  text-decoration: underline;  
}
```

Security Best Practices in Django

Learning goals

- Understand what web security means in simple terms.
- Learn about Django's built-in protection systems:
 - ✓ CSRF (Cross-Site Request Forgery)
 - ✓ XSS (Cross-Site Scripting)
 - ✓ Password hashing
 - ✓ HTTPS & session security
- See examples of what *can go wrong* and how Django prevents it.

Security Best Practices in Django

Why Security Matters

Threat	Meaning	Example
CSRF	Trick a logged-in user to perform actions they didn't intend	A fake website sends a POST request to your /students/add/ page
XSS	Inject JavaScript into your page	A user types <script> tags in a comment box
Password Theft	Storing plain passwords can expose users	Database leaks show every user's password
HTTP vs HTTPS	Unencrypted traffic can be read or changed	Attackers steal cookies over public Wi-Fi

Security Best Practices in Django

Django's Built-in Security Shield

The good news: Django already protects you from all of this *by default*!

CSRF – Cross-Site Request Forgery What is it?

CSRF happens when a malicious site tricks a logged-in user into performing an unwanted action on your app.

Example:

- You're logged in to your classroom app.
- You visit a malicious page that silently sends:

```
<form
action="http://127.0.0.1:8000/students/add/"
method="post">
  <input type="hidden" name="name"
value="Hacker">
  <input type="hidden" name="email"
value="evil@example.com">
</form>
<script>document.forms[0].submit();</script>
```

Without protection, that would add a fake student automatically!

Security Best Practices in Django

Django's Built-in Security Shield

Django's solution

Whenever you use a form, Django requires a CSRF token – a secret, random code added to the form that proves the request came from your site.

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Add</button>
</form>
```

If you forget it, Django shows:
“403 Forbidden – CSRF verification failed.”

Always include `{% csrf_token %}` inside `<form>` tags that use POST.

Security Best Practices in Django

Django's Built-in Security Shield

Try It!

1) Remove `{% csrf_token %}` from your `add_student.html`.

2) Try to submit the form → you'll get the CSRF error.

3) Put it back → it works again.

Forbidden (403)

CSRF verification failed. Request aborted.

Help

Reason given for failure:
CSRF token missing.

In general, this can occur when there is a genuine Cross Site Request Forgery, or when [Django's CSRF mechanism](#) has not been used correctly. For POST forms, you need to ensure:

- Your browser is accepting cookies.
- The view function passes a request to the template's [render](#) method.
- In the template, there is a `{% csrf_token %}` template tag inside each POST form that targets an internal URL.
- If you are not using `CsrfViewMiddleware`, then you must use `csrf_protect` on any views that use the `csrf_token` template tag, as well as those that accept the POST data.
- The form has a valid CSRF token. After logging in in another browser tab or hitting the back button after a login, you may need to reload the page with the form, because the token is rotated after a login.

You're seeing the help section of this page because you have `DEBUG = True` in your Django settings file. Change that to `False`, and only the initial error message will be displayed.

You can customize this page using the `CSRF_FAILURE_VIEW` setting.

Security Best Practices in Django

Django's Built-in Security Shield

XSS – Cross-Site Scripting

What is it?

XSS lets an attacker insert JavaScript into your page.

If you display user input directly, they can run malicious code in other users' browsers.

Example:

Someone adds a “student” named:

```
<script>alert("Hacked!")</script>
```

If you render it unsafely:

```
{{ student.name|safe }}
```

It executes the script!

Solution:

By default, Django escapes unsafe characters – `<`, `>`, `"` – turning them into harmless text.

Security Best Practices in Django

Django's Built-in Security Shield

XSS – Cross-Site Scripting



Safe:

```
<p>{{ student.name }}</p>
```



Dangerous:

```
<p>{{ student.name|safe }}</p>
```

Never use `|safe` unless you absolutely trust the data.

Security Best Practices in Django

Django's Built-in Security Shield

Password Hashing

What is it?

Hashing = a one-way transformation of a password into a long random string.

Django never stores plain text passwords.

Example:

If your password is **apple123**, Django stores:

`pbkdf2_sha256$6000000$FX8fM4PrA9xM$8QH...`

Even if hackers steal your database, they cannot reverse it to apple123.

Django uses PBKDF2 by default, which adds a unique “salt” and performs thousands of hash rounds to slow down attackers.

RegUser

Username:

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

algorithm: pbkdf2_sha256 **iterations:** 1000000 **salt:** 3AG70Z***** **hash:** EYojOp*****

[Reset password](#)

Raw passwords are not stored, so there is no way to see the user's password.

Security Best Practices in Django

Django's Built-in Security Shield

HTTPS and Secure Sessions

HTTP = unencrypted text traveling over the network.

HTTPS = encrypted connection via SSL/TLS.

Without HTTPS:

- ✓ Anyone on the same Wi-Fi can read your session cookie (the “key” that keeps you logged in).
- ✓ Attackers can impersonate you.

With HTTPS:

- ✓ All communication between your browser and server is private.
- ✓ Django works fine with HTTPS – you will configure it when deploying to Heroku or Azure.